ACADEMIC & TECHNICAL PROJECT REVIEW GUIDE

PriceRadar: Team Roles, Code Architecture & Viva Defense Manual

Individual Contribution Analysis, Architectural Source Code Breakdowns, Presentation Scripts, and Comprehensive 300+ Technical Viva Voce Question Bank across All Five Engineering Tracks.

Mrunal
Frontend Architect
React, Vite, State, Theme, OfferRow, Alerts, UX

Rahul
Backend & Auth
Express, JWT, Bcrypt, OTP, Input Validation, Security

Yaswanth
Database & Search
Mongoose, Integer Paise, Price Matrix, Aggregations

Vamshi
Integrations & Cloud
Apify, eBay API, Scrapling, Docker, Render PaaS

Ritesh
Monitoring & Alerts
Distributed Lease, Email, Telegram, GitHub Cron, CI

Project: Autonomous E-Commerce Price Monitoring System | **Institution Review:** Final Year B.Tech / BE Capstone Examination
Verification Baseline: 25 Jest Test Suites (263 Passed), 100% Passing Production Build, Zero Open Handles

1. Team Roles & Architectural Handoff Matrix

PriceRadar was engineered following strict separation of concerns among five specialized engineering roles. The table below details individual ownership, primary deliverables, and exact interface contracts between team members.

Member	Assigned Module	Core File Ownership	Architectural Handoff	Key Success Metric
Mrunal	Frontend Engineering & UI/UX	`client/src/App.jsx` `client/src/services/api.js` `OfferRow.jsx`, `AlertForm.jsx` `theme.css`, `ProductDetail.jsx`	Consumes REST APIs exposed by Rahul and Yaswanth; provides JWT in `Authorization` header; manages UI state.	Responsive UI, zero layout shift, instantaneous search filtering, resilient error fallbacks.
Rahul	Backend APIs, Auth & Security	`server/src/server.js`, `app.js` `controllers/authController.js` `middleware/auth.js`, `validate.js` `models/User.js`, `OtpChallenge.js`	Validates and secures requests from Mrunal; populates `req.user` for Yaswanth and Ritesh controllers.	JWT HMAC-SHA256 verification, bcrypt password hashing (10 salt rounds), rate limiting, zero leaking stack traces.
Yaswanth	Database & Product Catalog	`models/Product.js`, `Offer.js` `models/PriceObservation.js` `controllers/productsController.js` `services/priceComparison.js`	Provides normalized product data and deal rankings to Mrunal; stores scraped snapshots from Vamshi.	Integer Paise math (no float drift), compound indexes (`slug`, `domain_1_url_1`), atomic upserts.
Vamshi	Retailer Integrations & Cloud Infrastructure	`services/scrapers/apify.js` `services/scrapers/ebay.js` `services/scrapers/scrapling.js` `worker/main.py`, `Dockerfile`	Extracts multi-retailer live prices; feeds raw payload to Yaswanth's schema normalization pipeline; configures Render.	Anti-bot bypass via Scrapling Stealth + Apify cloud, resilient retailer fallbacks, sub-2s eBay API latency.
Ritesh	Monitoring Engine, Alerts & CI/CD	`services/priceMonitor.js` `services/alertEvaluator.js` `services/notifications/email.js` `.github/workflows/monitor.yml`	Executes 4-hour background cycle using Vamshi's scrapers; evaluates against Yaswanth's Watchlist; dispatches Brevo & Telegram.	Atomic distributed locking (`MonitorLock`), zero duplicate alerts (dedup key), 25 Jest CI test suites passing.

Architectural Contract Boundaries

To prevent integration conflicts and regression bugs during parallel development, the team established three rigid contracts:

- **Currency Precision Contract:** No float values are ever transmitted or persisted for currency. All values are integer Paise (1 INR = 100 Paise). The frontend formats for display using `formatCurrency(paise / 100)`.
- **Asynchronous Monitoring Contract:** The scheduled monitoring endpoint `POST /api/monitor/run` immediately returns `HTTP 202 Accepted` with a unique `runId`. The long-running scrape runs in the background guarded by a distributed database lease.
- **Authentication Token Contract:** The client stores JWT in `localStorage` and attaches it via Axios interceptor as `Bearer <token>`. Expired tokens receive `401 Unauthorized` and trigger seamless client-side logout.

Part 1: Mrunal — Frontend Development, API Integration & User Experience

Specialization: React 18, Vite, Single Page Application Architecture, State Management, UI Performance

1.1 Assigned Responsibilities & Deliverables

Mrunal was responsible for building the complete client-side architecture of PriceRadar using React 18 and Vite. His deliverables include the responsive layout engine, token-based authentication state management, REST client integration with Axios, product search and detail visualization, interactive price comparison tables, and multi-retailer alert management dialogs.

1.2 Core Code Breakdown & Snippets

A. API Client Interceptors (`client/src/services/api.js`)

Centralized Axios instance configured with base URLs and automatic request/response interceptors to manage JWT authorization headers and handle authentication expiration.

```
// client/src/services/api.js - Centralized Axios Client
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
  headers: { 'Content-Type': 'application/json' }
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
export default api;
```

Architectural Logic: Every outgoing request seamlessly acquires the Bearer token without manual component injection. If the backend invalidates the token, the 401 response interceptor evicts the stale credentials and redirects the user to the login screen, eliminating broken session states.

B. Multi-Retailer Offer Comparison Component (`client/src/components/OfferRow.jsx`)

Renders live comparison rows across Amazon, Flipkart, Croma, Reliance Digital, and eBay, computing difference deltas and badging the lowest available price.

```
// client/src/components/OfferRow.jsx - Best Deal Calculation & Display
export default function OfferRow({ offer, isLowest, targetPricePaise }) {
  const displayPrice = (offer.priceInPaise / 100).toLocaleString('en-IN', {
    style: 'currency',
    currency: 'INR',
    maximumFractionDigits: 0
  });
```

```
  return (
    <div className={`offer-row ${isLowest ? 'offer-lowest' : ''}`}>
      <div className="retailer-badge">{offer.retailer}</div>
      <div className="price-display">
        <span className="price-current">{displayPrice}</span>
        {isLowest && <span className="best-deal-tag">Lowest Price</span>}
      </div>
      <a href={offer.affiliateUrl || offer.productUrl} target="_blank" rel="noreferrer">
        Buy on {offer.retailer}
      </a>
    </div>
  );
}
```

1.3 Mrunal's Presentation Scripts

2-MINUTE PRESENTATION SCRIPT (SPOKEN IN REVIEW)

"Good morning evaluators. I am Mrunal, and I built the client-side architecture and user interface for PriceRadar using React 18 and Vite. My objective was to deliver a responsive, zero-latency price tracking dashboard that effortlessly handles multi-retailer comparisons. I engineered our API client layer with Axios interceptors that inject JWT tokens into all secured endpoints and gracefully handle session invalidation. In the product views, I implemented dynamic price matrix tables with real-time lowest-price badging, historical trend visualization with Chart.js, and modal forms for configuring custom alert thresholds. Every currency value is passed as integer Paise to maintain strict numerical accuracy. Our production build is compiled with Vite for sub-second hot reload and highly optimized production bundles."

5-MINUTE VIVA TECHNICAL DEFENSE SCRIPT

"When designing the frontend for PriceRadar, our primary architectural challenge was dealing with asynchronous multi-retailer state. Unlike simple CRUD applications, price tracking data changes dynamically and involves complex calculations like target price comparisons and savings percentages. I chose React 18 with Vite over Create React App because Vite's native ES module tooling provides near-instantaneous build and refresh cycles. For state management, I utilized React Context for global authentication state combined with local component hooks for page-level data fetching. Our Axios interceptor setup acts as a protective boundary: it handles outgoing Bearer headers automatically, while our response interceptor intercepts 401 Unauthorized responses to purge expired tokens from local storage and route cleanly to the login view. In the user interface, we enforce strict separation between data representation and display formatting: all price data is transmitted across the wire in integer Paise, and only formatted into Indian Rupee strings using the Intl.NumberFormat API at the presentation boundary."

1.4 Mrunal's 50 Viva Voce Questions & Answers

BASIC **What is Vite and why did you choose it over Create React App (CRA)?**

Vite uses native ES modules in development and Rollup for production bundling. CRA relies on Webpack, which bundles the entire application before serving, leading to slow startup times. Vite provides sub-second hot module replacement (HMR) and significantly smaller production bundle sizes.

BASIC **What is JSX in React?**

JSX (JavaScript XML) is a syntax extension that allows writing HTML-like structure directly within JavaScript. Babel/esbuild compiles JSX into standard `React.createElement()` function calls.

BASIC **What are React Hooks and why are they used?**

Hooks (`useState`, `useEffect`, `useContext`) are functions that allow functional components to maintain state and tap into lifecycle events without writing class components.

BASIC **What is the Virtual DOM and how does React update the browser?**

The Virtual DOM is an in-memory lightweight representation of the actual DOM. When component state changes, React creates a new virtual tree, diffs it with the previous tree using its reconciliation algorithm, and updates only the changed DOM nodes.

BASIC **What is the purpose of `localStorage` in PriceRadar?**

We store the user's JWT token and theme preference in `localStorage`. This allows user authentication sessions to persist across browser refreshes and tab reopens.

BASIC **Why do we need Axios interceptors?**

Interceptors run before requests leave or after responses return. In PriceRadar, request interceptors attach the `Bearer ` to all HTTP requests, and response interceptors intercept `401 Unauthorized` errors to force clean logouts.

BASIC **What is Single Page Application (SPA) architecture?**

In an SPA, the browser loads a single HTML page once. Navigation between routes is managed on the client side using `react-router-dom` without full page reloads, providing a desktop-like experience.

BASIC **What does `useEffect` do when passed an empty dependency array `[]`?**

It executes the effect callback only once after the initial component mount, behaving similarly to `componentDidMount` in class components.

BASIC **How do you conditionally render elements in React?**

Using ternary operators (`condition ? : `), logical AND short-circuiting (`condition && `), or standard `if/else` statements in helper render functions.

BASIC **What is prop drilling and how do you prevent it?**

Prop drilling occurs when data is passed through multiple intermediate components that do not need it. We prevent it using React Context (`AuthContext`) to share authentication state globally.

BASIC **What is the difference between controlled and uncontrolled components?**

Controlled components have their input values driven by React state via `value` and `onChange` props. Uncontrolled components store their values directly in the DOM and are accessed via `ref`.

BASIC **Why must React list items have a `key` prop?**

Keys give elements a stable identity across renders. React uses keys during the Virtual DOM diffing process to identify which items were added, modified, or removed, preventing unnecessary re-renders.

BASIC **What is the purpose of `React.Fragment` or `<>`?**

It allows returning multiple child elements without creating extra wrapping nodes in the real DOM, keeping the DOM tree clean.

BASIC **How does CSS Modules or scoped CSS prevent styling conflicts?**

By scoping class names locally to individual components using generated unique hashes, preventing global CSS name collisions.

BASIC **What is the difference between synchronous and asynchronous code in JavaScript?**

Synchronous code executes sequentially and blocks subsequent operations. Asynchronous code (Promises, async/await) allows non-blocking execution while waiting for external I/O like network requests.

INTERMEDIATE **How does `AuthContext` manage login and logout across the application?**

We wrap the app in an `AuthProvider` containing `user` state and `token`. When `login()` is called, the JWT is stored in `localStorage` and decoded to populate `user`. When `logout()` is called, `localStorage` is cleared and `user` is set to null, immediately updating all dependent UI routes.

INTERMEDIATE **How do protected routes work in `client/src/App.jsx`?**

We create a `ProtectedRoute` wrapper component that checks if `isAuthenticated` is true. If true, it renders `` or the child component; otherwise, it redirects the visitor to `/login` via ``.

INTERMEDIATE **How does `OfferRow` highlight the best deal?**

The parent component computes the minimum `priceInPaise` across all offers and passes an `isLowest={offer.priceInPaise === minPrice}` boolean prop. The row conditionally appends the `offer-lowest` CSS class and renders the 'Lowest Price' badge.

INTERMEDIATE **Why do we convert paise to rupees on the client instead of receiving rupees from the backend?**

To avoid IEEE 754 floating-point inaccuracies during arithmetic operations. The backend guarantees integer precision, and the client applies localized currency formatting solely at the render boundary using `Intl.NumberFormat`.

INTERMEDIATE **How does the search debouncing work in the product catalog?**

We use a custom `useDebounce` hook or a `setTimeout` inside `useEffect` that delays sending the search query to the API until 300ms after the user stops typing, preventing excessive network traffic.

INTERMEDIATE **How do you handle loading and error states during API calls?**

Each page maintains `loading` and `error` states. Before the async call, `setLoading(true)` and `setError(null)` are invoked. In the `try/catch/finally` block, exceptions update `error`, and `finally` resets `loading(false)`, rendering spinners or alert banners accordingly.

INTERMEDIATE **How is responsive design implemented across mobile and desktop?**

Through modern CSS flexbox, CSS grid layouts, and media queries defined in `theme.css`. Tables collapse into vertical card layouts on viewports narrower than 768px.

INTERMEDIATE **What is the purpose of `targetPricePaise` validation in `AlertForm.jsx`?**

It prevents users from setting negative or zero alert thresholds, and warns them if their target price is greater than the current lowest price, ensuring meaningful alert triggers.

INTERMEDIATE **How does `PriceChart.jsx` render price trends?**

It uses Chart.js with `react-chartjs-2`. It maps historical `priceObservations` with timestamps on the X-axis and `priceInPaise / 100` on the Y-axis, rendering multi-retailer line graphs.

INTERMEDIATE **What happens if an image URL fails to load in a product card?**

We attach an `onError` event handler to the `` tag that swaps the broken URL with a local SVG placeholder image, preventing broken image icons.

INTERMEDIATE **How do you prevent memory leaks when unmounting components with active timers or API calls?**

In `useEffect`, we return a cleanup function that cancels pending `setTimeout` calls or aborts ongoing fetch requests using `AbortController`.

INTERMEDIATE **How do modal dialogs trap keyboard focus and dismiss on Escape?**

We attach a `keydown` listener to the `window` checking for `event.key === 'Escape'` and wrap modal content with backdrop click handlers that trigger the close callback.

INTERMEDIATE **What is the difference between `useMemo` and `useCallback`?**

`useMemo` caches the calculated result of an expensive function between renders, whereas `useCallback` caches the function instance itself to prevent unnecessary child re-renders.

INTERMEDIATE **How are environment variables loaded in Vite?**

Vite exposes environment variables on `import.meta.env`. Only variables prefixed with `VITE_` (e.g., `VITE_API_URL`) are exposed to client-side code for security.

INTERMEDIATE **How does PriceRadar support Dark Mode?**

CSS custom properties (variables) defined in `:root` and `[data-theme='dark']` in `theme.css`. Toggling the theme switches the attribute on `document.documentElement` and stores the choice in `localStorage`.

ADVANCED **How would you optimize the React app for Core Web Vitals (LCP, FID, CLS)?**

1) LCP: Preload hero product images and use code splitting (`React.lazy`). 2) FID: Minimize main-thread blocking by debouncing search inputs. 3) CLS: Reserve explicit width/height aspect-ratio containers for product images and dynamic offer tables.

ADVANCED **Explain React 18 Concurrent Rendering and Automatic Batching.**

React 18 batches multiple state updates within promises, timeouts, and native event handlers into a single re-render cycle. Concurrent rendering allows React to interrupt rendering long lists if user interactions (like typing) take priority.

ADVANCED **How would you implement Infinite Scrolling for product search?**

Using the `IntersectionObserver` API. An invisible sentinel element at the bottom of the list triggers a fetch for the next page (`page + 1`) when it enters the viewport, appending new products to the existing state array.

ADVANCED **How does client-side routing with `react-router-dom` work under the hood?**

It uses the HTML5 History API (`pushState`, `replaceState`, and `popstate` events) to change the browser URL and update the view hierarchy without requesting a new HTML page from the web server.

ADVANCED **Why should sensitive data never be stored in `localStorage`?**

Any JavaScript running on the page (including third-party scripts or XSS exploits) can access `localStorage`. We mitigate this by storing only the JWT, ensuring tokens have short expiration, and escaping user input.

ADVANCED **How would you migrate from `localStorage` to HttpOnly cookies for JWTs?**

The backend would set the JWT in a `Set-Cookie: token=...; HttpOnly; Secure; SameSite=Strict` header. The browser automatically attaches it to subsequent requests, making the token inaccessible to JavaScript and immune to XSS theft.

ADVANCED **What is Tree Shaking and how does Vite achieve it?**

Tree shaking removes unused export statements from the final bundle. Because ES modules (`import`/`export`) are statically analyzable, Rollup eliminates dead code during the production build.

ADVANCED **How do you debug rendering bottlenecks in a complex React tree?**

Using React Developer Tools Profiler to inspect component render durations, commit phases, and the exact props that triggered re-renders.

ADVANCED **Explain the trade-offs of Client-Side Rendering (CSR) vs Server-Side Rendering (SSR) for PriceRadar.**

CSR provides fast subsequent navigation and lower server compute costs, but suffers from slower initial First Contentful Paint. SSR (like Next.js) renders HTML on the server, boosting SEO for public product pages at the expense of server load.

ADVANCED **How do you prevent cross-site scripting (XSS) in React components?**

React escapes values embedded in JSX expressions by default before rendering them to strings. We avoid using `dangerouslySetInnerHTML` unless sanitizing inputs through libraries like DOMPurify.

CODE **In `client/src/services/api.js`, what happens if the backend returns a 401 error?**

The Axios response interceptor catches `error.response?.status === 401`, executes `localStorage.removeItem('token')`, and forces a window redirect to `/login` to reset auth state.

CODE **In `OfferRow.jsx`, how is `displayPrice` formatted?**

It divides `offer.priceInPaise` by 100 and applies `.toLocaleString('en-IN', { style: 'currency', currency: 'INR', maximumFractionDigits: 0 })` for clean Indian Rupee presentation.

CODE **What props does `OfferRow` accept?**

`offer` (offer object with retailer, price, URLs), `isLowest` (boolean indicating if it is the cheapest deal), and `targetPricePaise` (user's target price for comparison).

CODE **How does `AlertForm.jsx` submit a new price alert?**

It collects the retailer selection and target price, converts rupees to paise (`Math.round(rupees * 100)`), and sends a `POST /api/alerts` request via `api.post()`.

CODE **How does `ProductDetail.jsx` fetch product data?**

It extracts the `:id` parameter using `useParams()`, calls `api.get('/products/' + id)` inside `useEffect`, and populates local `product` state.

CODE **How does `Watchlist.jsx` allow users to remove items?**

It triggers `api.delete('/watchlist/' + id)`, filters out the deleted item from the local array state, and displays a confirmation toast notification.

CODE **Where are the CSS variables for the color palette defined?**

In `client/src/styles/theme.css` under the `:root` pseudo-class, defining primary blues, background slate shades, text colors, and border tokens.

CODE **In `client/src/App.jsx`, which routes are public and which are protected?**

Public: `/`, `/login`, `/register`, `/products/:id`. Protected: `/watchlist`, `/alerts`, `/dashboard`, which require a valid JWT.

CODE **How does `PriceChart.jsx` prevent canvas re-use errors in Chart.js?**

`react-chartjs-2` handles Chart.js instance destruction and canvas lifecycle automatically within its unmount hook.

CODE **What command builds the frontend for production deployment?**

`npm run build` in the `client/` directory, which executes `vite build` to output optimized static assets into `client/dist/`.

Part 2: Rahul — Backend APIs, Authentication, Authorization & Security

Specialization: Node.js, Express, JWT Security, Password Hashing, Input Validation, Middleware Chains

2.1 Assigned Responsibilities & Deliverables

Rahul architected the backend server infrastructure, secure authentication pipelines, and API security boundary. His deliverables include the Express application lifecycle, MongoDB database connection manager, JWT generation and verification middleware, bcrypt password hashing, OTP challenge generation and verification, request body validation schemas, centralized error handling, and route protection.

2.2 Core Code Breakdown & Snippets

A. JWT Authentication Middleware (`server/src/middleware/auth.js`)

Intercepts requests to protected endpoints, parses Bearer authorization headers, verifies the cryptographic signature, and injects user context.

```
// server/src/middleware/auth.js - Bearer Token Verification
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const auth = async (req, res, next) => {
  try {
    const authHeader = req.header('Authorization');
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ success: false, message: 'No token provided' });
    }

    const token = authHeader.replace('Bearer ', '').trim();
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    const user = await User.findById(decoded.id).select('-passwordHash');
    if (!user) {
      return res.status(401).json({ success: false, message: 'User not found or deleted' });
    }

    req.user = user;
    next();
  } catch (err) {
    return res.status(401).json({ success: false, message: 'Invalid or expired token' });
  }
};
module.exports = auth;
```

Security Review: The middleware extracts the raw token, verifies its signature against `process.env.JWT_SECRET`, and ensures the user still exists in the database while stripping out the password hash before assigning to `req.user`.

B. Password Hashing & Registration (`server/src/controllers/authController.js`)

Enforces password complexity, hashes secrets using bcrypt with 10 salt rounds, and issues signed JWT tokens upon successful registration.

```
// server/src/controllers/authController.js - Secure Registration
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');

exports.register = async (req, res, next) => {
  try {
    const { name, email, password } = req.body;
    const existing = await User.findOne({ email: email.toLowerCase().trim() });
    if (existing) {
```

```
      return res.status(409).json({ success: false, message: 'Email already registered' });
    }

    const salt = await bcrypt.genSalt(10);
    const passwordHash = await bcrypt.hash(password, salt);

    const user = await User.create({
      name: name.trim(),
      email: email.toLowerCase().trim(),
      passwordHash
    });

    const token = jwt.sign({ id: user._id, email: user.email }, process.env.JWT_SECRET, { expiresIn: '7d' });
    res.status(201).json({
      success: true,
      token,
      user: { id: user._id, name: user.name, email: user.email }
    });
  } catch (err) {
    next(err);
  }
};
```

2.3 Rahul's Presentation Scripts

2-MINUTE PRESENTATION SCRIPT (SPOKEN IN REVIEW)

"Good morning evaluators. I am Rahul, and I engineered the backend server architecture, authentication protocols, and security infrastructure for PriceRadar. I built our Node.js and Express REST API service with a clean modular structure separating routes, controllers, middleware, and domain models. I implemented stateless JWT authentication using HMAC-SHA256 tokens signed with high-entropy secrets and protected all user data with bcrypt password hashing using 10 salt rounds. To secure our platform, I built custom middleware for input validation, Bearer token verification, role-based authorization, and centralized error handling that eliminates production stack trace leaks. Our backend maintains clean database connection lifecycles, and every route is validated with automated Jest integration tests."

5-MINUTE VIVA TECHNICAL DEFENSE SCRIPT

"In architecting PriceRadar's backend, my principal objective was building a bulletproof security perimeter and clean request pipeline. We chose Express on Node.js for its high-throughput asynchronous event loop and vast middleware ecosystem. For authentication, we chose JSON Web Tokens over server-side session stores like Redis to keep our API stateless, allowing horizontal scaling across multiple container instances without shared session state. When a user registers or logs in, their credentials pass through our validation middleware. Passwords are never stored in plaintext: we generate a random 10-round salt and compute an irreversible bcrypt hash. Our `auth` middleware protects private endpoints: it parses the Authorization header, validates the signature against our environment secret, checks token expiration, and loads the user record from MongoDB while explicitly suppressing the passwordHash field using `.select('-passwordHash')`. To prevent unhandled promise rejections from crashing the Node.js process, all asynchronous controller functions forward errors to our centralized error handler via `next(err)`, which returns structured JSON responses with appropriate HTTP status codes."

2.4 Rahul's 50 Viva Voce Questions & Answers

BASIC **What is Node.js and how does its event loop work?**

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine. Its event loop offloads I/O operations to the system kernel (via libuv), executing non-blocking callbacks on a single thread.

BASIC **What is Express.js?**

Express is a minimal and flexible Node.js web application framework that provides robust routing, middleware support, and HTTP utility methods.

BASIC **What is a RESTful API?**

A Representational State Transfer API uses standard HTTP methods (GET, POST, PUT, DELETE) to manipulate resources identified by URIs, following stateless client-server conventions.

BASIC **What is JWT (JSON Web Token)?**

JWT is an open standard (RFC 7519) for securely transmitting information between parties as a compact JSON object, consisting of Header, Payload, and Signature.

BASIC **What is bcrypt and why is it preferred over SHA-256 for passwords?**

Bcrypt is an adaptive key derivation function with an adjustable work factor (salt rounds). SHA-256 is designed for fast hashing, making it vulnerable to GPU brute-force attacks, whereas bcrypt is deliberately slow and salted.

BASIC **What is Middleware in Express?**

Middleware functions have access to the request (`req`), response (`res`), and `next` function, allowing them to modify requests, run checks, or terminate request cycles.

BASIC **What is the difference between `npm start` and `node server.js`?**

`node server.js` invokes Node directly. `npm start` executes the script defined in `package.json`, which may include environment flags, nodemon, or pre-start build steps.

BASIC **What is CORS and why is it required?**

Cross-Origin Resource Sharing is a browser security mechanism that restricts web pages from making API requests to a different domain/port than the one serving the web page.

BASIC **What does `res.status(201).json()` signify?**

HTTP 201 indicates that a new resource was successfully created on the server, accompanied by a JSON representation of that resource.

BASIC **What is the purpose of `dotenv`?**

It loads environment variables from a `.env` file into `process.env` during development, keeping credentials out of source code.

BASIC **What is the difference between `req.params`, `req.query`, and `req.body`?**

`req.params` contains route parameters (e.g., `/products/:id`). `req.query` contains URL query string parameters (e.g., `?search=macbook`). `req.body` contains parsed JSON payload data.

BASIC **Why do we convert user emails to lowercase before saving?**

Email addresses are case-insensitive. Normalizing them to lowercase prevents duplicate user accounts for `User@Example.com` and `user@example.com`.

BASIC **What is rate limiting and why is it needed?**

Rate limiting restricts the number of requests a client IP can make in a given timeframe, protecting APIs against brute-force attacks and denial-of-service.

BASIC **What does `next(err)` do in Express?**

It bypasses all remaining regular middleware and passes control directly to the registered 4-argument error handling middleware (`(err, req, res, next)`).

BASIC **What is an OTP challenge?**

A One-Time Password challenge generates a time-sensitive numerical code stored with a TTL (Time-To-Live) to verify email ownership before sensitive actions.

INTERMEDIATE **How does the `auth` middleware prevent unauthorized access?**

It extracts `Authorization: Bearer `, verifies the HMAC-SHA256 signature using `jwt.verify()`, looks up the user in MongoDB, and attaches it to `req.user`. Missing or invalid tokens immediately yield HTTP 401.

INTERMEDIATE **Why do we exclude `passwordHash` using `.select('-passwordHash')` in `auth.js`?**

To prevent the hashed password string from ever leaking into memory structures, downstream controllers, or inadvertently into API JSON responses.

INTERMEDIATE **What is a salt round in bcrypt and why did we choose 10?**

The salt round determines the number of hashing iterations ($2^{10} = 1024$ rounds). 10 provides strong cryptographic protection against brute-force while hashing in ~80ms on standard CPUs without blocking the event loop.

INTERMEDIATE **How does centralized error handling prevent production information leaks?**

In `errorHandler.js`, we inspect `process.env.NODE_ENV`. If 'production', stack traces are omitted, returning only `{ success: false, message: 'Internal Server Error' }`.

INTERMEDIATE **How does MongoDB injection differ from SQL injection and how do you prevent it?**

Attackers send objects like `{ $gt: '' }` instead of strings to bypass auth checks. We prevent this using `express-mongo-sanitize` and strict Mongoose schema validation.

INTERMEDIATE **How does `validate.js` validate incoming request payloads?**

It checks required fields, email format regex, password minimum lengths, and positive numbers, returning HTTP 400 with specific field error messages if validation fails.

INTERMEDIATE **What is the life cycle of a JWT token in PriceRadar?**

Issued at `/api/auth/login` or `/register` with a 7-day expiration (`expiresIn: '7d'`). Stored in client `localStorage`, transmitted in Bearer headers, and validated statelessly on each request.

INTERMEDIATE **How does `server/src/config/db.js` handle database reconnections?**

It connects via `mongoose.connect()` and attaches event listeners to `mongoose.connection.on('error')` and `disconnected`, implementing exponential backoff reconnection logic.

INTERMEDIATE **Why is `server.js` separated from `app.js`?**

`app.js` configures Express routes, middleware, and settings without starting the server, enabling Jest integration tests to import `app` directly via Supertest without binding to live TCP ports.

INTERMEDIATE **How do you handle graceful shutdown in Node.js?**

By listening to `SIGINT` and `SIGTERM` signals, calling `server.close()` to stop accepting new requests, and invoking `mongoose.connection.close()` before exiting with `process.exit(0)`.

INTERMEDIATE **What is Helmet.js and how does it protect the application?**

Helmet sets secure HTTP response headers such as Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Strict-Transport-Security to thwart common web vulnerabilities.

INTERMEDIATE **How does the OTP verification system expire old tokens?**

The `OtpChallenge` schema defines an `expiresAt` field with a MongoDB TTL index: `{ expireAfterSeconds: 0 }`. MongoDB automatically deletes expired challenges in the background.

INTERMEDIATE **What is the difference between authentication and authorization?**

Authentication verifies identity ('who you are' via credentials/token). Authorization verifies permissions ('what you are allowed to access' via roles like Admin or Owner).

INTERMEDIATE **How are CORS preflight `OPTIONS` requests handled?**

The browser sends an `OPTIONS` request before non-simple cross-origin requests. The `cors()` middleware automatically responds with allowed methods, headers, and origins.

INTERMEDIATE **Why must `app.use(express.json())` appear before route definitions?**

It parses incoming JSON request bodies into JavaScript objects on `req.body`. If routes are placed before it, `req.body` will be `undefined`.

ADVANCED **How would you implement Refresh Tokens to enhance security?**

Issue a short-lived access token (15 mins) and a long-lived refresh token (30 days) stored in an HttpOnly cookie and database whitelist. When the access token expires, the client hits `/api/auth/refresh` to obtain a fresh access token without re-entering credentials.

ADVANCED **What is the difference between symmetric and asymmetric JWT signing?**

Symmetric (HS256) uses a single shared secret key to both sign and verify. Asymmetric (RS256) uses a private key on the auth server to sign and a public key on resource servers to verify, allowing distributed verification.

ADVANCED **How do you prevent Denial of Service (DoS) attacks on the regex engine (ReDoS)?**

Avoid complex nested quantifier regexes (e.g., `(a+)+$`). Use simple email regexes, test with safe-regex tools, or limit maximum string input length before evaluation.

ADVANCED **How does Node.js cluster module or PM2 scale Express across multiple CPU cores?**

Because Node is single-threaded, cluster/PM2 forks child worker processes that share the same server port. The master process load-balances incoming TCP connections using round-robin.

ADVANCED **How would you implement Role-Based Access Control (RBAC)?**

Add a `role` field to `User` schema (`['user', 'admin']`). Create a middleware `authorize('admin')` that inspects `req.user.role` after `auth` and rejects with HTTP 403 Forbidden if permissions are insufficient.

ADVANCED **How do you prevent timing attacks during password verification?**

Standard string comparison (`===`) terminates on the first mismatch, allowing attackers to measure time differences. `bcrypt.compare()` uses constant-time byte comparisons to prevent timing analysis.

ADVANCED **What is the risk of Server-Side Request Forgery (SSRF) in PriceRadar and how is it mitigated?**

If users provide arbitrary URLs to track, an attacker could input `http://169.254.169.254/latest/meta-data` to steal cloud metadata. We mitigate this by validating URLs against an allowlist of supported retailer domains.

ADVANCED **Explain the difference between vertical scaling and horizontal scaling for Node.js.**

Vertical scaling increases CPU/RAM of the existing host machine. Horizontal scaling provisions multiple stateless server instances behind a load balancer (e.g., Nginx, AWS ALB).

ADVANCED **How do you secure environment secrets in production CI/CD?**

Secrets are injected via GitHub Actions Encrypted Secrets and Render Environment Variables. They are never written to disk, committed to Git, or logged to console outputs.

ADVANCED **How does Express handle memory leaks from unclosed streams or sockets?**

Unclosed streams keep event listeners active in memory. We attach `finish` or `close` listeners to pipe streams safely and use tools like Clinic.js or Chrome DevTools heap snapshots to detect detached objects.

CODE **In `server/src/middleware/auth.js`, what does `authHeader.replace('Bearer ', '').trim()` accomplish?**

It strips the `Bearer ` prefix and any trailing whitespace from the HTTP Authorization header string to isolate the raw JWT token.

CODE **In `server/src/controllers/authController.js`, why is `email.toLowerCase().trim()` executed twice?**

Once during duplicate user lookup (`User.findOne`) and once during user creation (`User.create`) to ensure case-insensitive email uniqueness.

CODE **What HTTP status code does `authController.register` return when the email already exists?**

HTTP 409 Conflict, indicating that the request could not be completed due to a conflict with existing database records.

CODE **In `server/src/app.js`, what does `app.use(express.urlencoded({ extended: true }))` do?**

It parses incoming URL-encoded form data payloads into `req.body`, with `extended: true` permitting rich objects and arrays via the `qs` library.

CODE **What is the function of `server/src/middleware/validate.js`?**

It acts as a gatekeeper middleware validating request schema fields before controllers execute, returning standardized HTTP 400 Bad Request error payloads.

CODE **In `server/src/models/User.js`, what does `trim: true` on fields enforce?**

Mongoose automatically strips leading and trailing whitespace characters from strings before saving to MongoDB.

CODE **How does `server/src/server.js` bind to the environment port?**

`const PORT = process.env.PORT || 5000; app.listen(PORT, ...)` allows dynamic port binding in cloud environments like Render.

CODE **In `server/src/middleware/errorHandler.js`, how are Mongoose validation errors handled?**

It detects `err.name === 'ValidationError'`, extracts error messages from each schema path, and returns HTTP 400 with a clean list of validation failures.

CODE **How does `server/src/routes/watchlist.js` protect its routes?**

It applies `router.use(auth)` at the top of the route file, ensuring all watchlist CRUD operations require a valid JWT.

CODE **What does `disconnectDB()` in `server/src/config/db.js` do during test runs?**

It executes `await mongoose.disconnect()`, terminating all active connection pools so Jest can exit cleanly with zero open handles.

Part 3: Yaswanth — Database Design, Product Persistence, Search & Price Analytics

Specialization: MongoDB Atlas, Mongoose Schemas, Integer Paise Arithmetic, Compound Indexes, Aggregation Pipelines

3.1 Assigned Responsibilities & Deliverables

Yaswanth engineered the database schemas, entity relationships, catalog indexing strategies, and price comparison analytics. His deliverables include the Mongoose models for Products, Offers, PriceObservations, and WatchlistItems, integer currency normalization (`priceInPaise`), text search indexing, product aggregation queries, and deterministic database seeding for automated testing.

3.2 Core Code Breakdown & Snippets

A. Integer Currency Normalization (`server/src/utils/money.js`)

Eliminates floating-point rounding bugs by enforcing integer math for all monetary transactions.

```
// server/src/utils/money.js - Strict Integer Currency Representation
exports.toPaise = (rupees) => {
  if (typeof rupees !== 'number' || isNaN(rupees)) {
    throw new TypeError('Price must be a valid number');
  }
  return Math.round(rupees * 100);
};

exports.toRupees = (paise) => {
  if (!Number.isInteger(paise)) {
    throw new TypeError('Paise must be an integer');
  }
  return (paise / 100).toFixed(2);
};
```

Numerical Precision Rule: In standard IEEE 754 floating point arithmetic, `0.1 + 0.2 = 0.30000000000000004`. In e-commerce price monitoring, this drift can cause erroneous alerts. By storing ₹1,499.50 as `149950` integer Paise, price comparisons are mathematically exact.

B. Product & Offer Schema with Compound Indexes (`server/src/models/Product.js` & `Offer.js`)

Defines schema validations and compound indexes for sub-millisecond query performance.

```
// server/src/models/Offer.js - Compound Unique Index for Deduplication
const mongoose = require('mongoose');

const offerSchema = new mongoose.Schema({
  product: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true, index: true },
  retailer: { type: String, required: true, enum: ['Amazon', 'Flipkart', 'Croma', 'Reliance Digital', 'eBay'] },
  productUrl: { type: String, required: true },
  priceInPaise: { type: Number, required: true, min: 0 },
  inStock: { type: Boolean, default: true },
  lastCheckedAt: { type: Date, default: Date.now }
}, { timestamps: true });

// Prevent duplicate offer records for same product & retailer
offerSchema.index({ product: 1, retailer: 1 }, { unique: true });
module.exports = mongoose.model('Offer', offerSchema);
```

3.3 Yaswanth's Presentation Scripts

> **2-MINUTE PRESENTATION SCRIPT (SPOKEN IN REVIEW)**

> *"Good morning evaluators. I am Yaswanth, and I designed the database architecture, schema models, and price comparison analytics for PriceRadar. I modeled our persistent domain in MongoDB Atlas using Mongoose, establishing clear entity relationships between Products, Offers, Historical Price Observations, and User Watchlists. A foundational architectural decision I made was enforcing integer currency subunits: all prices are stored strictly in integer Paise, which completely eliminates IEEE 754 floating-point inaccuracies. I engineered compound database indexes—such as unique indexes on product and retailer pairs—to guarantee atomic upserts and sub-millisecond query execution. I also created our deterministic fixture seeding pipelines, ensuring that our automated test suites and production deployments run with verified baseline data."*

> **5-MINUTE VIVA TECHNICAL DEFENSE SCRIPT**
>
> *"In designing PriceRadar's data tier, the chief priority was ensuring absolute data integrity and high-performance querying under heavy read workloads. We selected MongoDB as our primary database because e-commerce product structures are inherently polymorphic: an electronics product from Amazon contains different attribute specifications than one from Flipkart. However, to avoid unconstrained schema sprawl, I used Mongoose to enforce strict schema boundaries. Our data model separates static catalog data in the `Product` collection from dynamic retailer pricing in the `Offer` collection. To track trends over time without inflating the active offer documents, every scraping cycle appends a snapshot to the `PriceObservation` collection. To achieve optimal query performance, I established specialized indexes: a unique text index on `name` and `slug` for instantaneous product searching, a compound unique index on `{ product: 1, retailer: 1 }` to make offer updates idempotent, and a TTL index on OTP challenges. For financial calculations, our `money.js` utility converts all currency to integer Paise, ensuring that alert trigger conditions are evaluated with zero rounding ambiguity."*

3.4 Yaswanth's 50 Viva Voce Questions & Answers

BASIC **What is MongoDB and how does it differ from relational databases like MySQL?**

MongoDB is a document-oriented NoSQL database that stores data in flexible JSON-like BSON documents. Relational databases store data in rigid tables with fixed columns and require schema migrations.

BASIC **What is Mongoose?**

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js, providing schema validation, type casting, middleware hooks, and business logic methods.

BASIC **What is a MongoDB Document and Collection?**

A document is a set of key-value pairs (equivalent to a row). A collection is a grouping of documents (equivalent to a table).

BASIC **What is an ObjectId in MongoDB?**

An ObjectId is a 12-byte BSON identifier composed of a 4-byte timestamp, 5-byte random value, and 3-byte incrementing counter, ensuring global uniqueness without centralized coordination.

BASIC **Why are timestamps (`createdAt`, `updatedAt`) useful in schemas?**

Setting `{ timestamps: true }` instructs Mongoose to automatically manage creation and modification dates, enabling audit trails and freshness sorting.

BASIC **What does `ref` mean in a Mongoose schema definition?**

It specifies the target Model name for population, creating a reference relationship analogous to a foreign key in relational databases.

BASIC **What is an Index in MongoDB?**

An index is a specialized B-tree data structure that holds a sorted portion of the collection's data to allow queries to find matching documents without scanning the entire collection.

BASIC **What happens during a Collection Scan (COLLSCAN)?**

MongoDB must sequentially inspect every single document in the collection to satisfy a query, causing severe CPU and disk I/O bottlenecks on large datasets.

BASIC **What is the purpose of `slug` in the Product schema?**

A slug is a URL-friendly string derived from the product title (e.g., `apple-iphone-15-black-128gb`), optimizing readability and search engine optimization.

BASIC **What does the `unique: true` constraint on a schema index enforce?**

It prevents MongoDB from inserting or updating documents if the indexed field value already exists in another document in the collection.

BASIC **What is `populate()` in Mongoose?**

It automatically replaces an ObjectId reference in a document with the actual referenced document from another collection during query execution.

BASIC **What is the difference between `findById()` and `findOne()`?**

`findById(id)` is a shorthand for `findOne({ _id: id })` specifically optimized for primary key lookups.

BASIC **What does `min: 0` in `priceInPaise` enforce?**

It prevents accidental persistence of negative prices at the schema validation level.

BASIC **What is BSON?**

BSON (Binary JSON) is the binary-encoded serialization format used by MongoDB to store documents and perform remote procedure calls, supporting data types like Date and raw binary.

BASIC **What is the purpose of database seeding?**

Populating a fresh database with realistic baseline records (products, offers, users) so developers and automated test suites can execute against known data structures.

INTERMEDIATE **Why did you store prices as integer Paise rather than Rupees?**

Floating-point representation in JavaScript follows IEEE 754, causing rounding anomalies (e.g., `0.1 + 0.2 !== 0.3`). Storing prices in integer Paise guarantees exact mathematical operations and prevents false alert evaluations.

INTERMEDIATE **How does the compound index `{ product: 1, retailer: 1 }` prevent data duplication?**

It enforces uniqueness across both fields simultaneously. A product can have offers from different retailers, but cannot have two conflicting offers for the same retailer, ensuring idempotent upserts.

INTERMEDIATE **Explain the difference between Embedded Documents and Referenced Documents.**

Embedded documents are nested directly inside a parent document (great for 1:few data that is always fetched together). Referenced documents store ObjectIds linking separate collections (ideal for 1:many or unbounded growth like price history).

INTERMEDIATE **Why is `PriceObservation` stored as a separate collection instead of an embedded array in `Product`?**

MongoDB has a 16MB document size limit. With price monitoring every 4 hours across 5 retailers, an embedded array would grow by thousands of entries per year, eventually hitting the 16MB limit and causing performance degradation.

INTERMEDIATE **How does `services/priceComparison.js` compute the lowest price for a product?**

It fetches all active in-stock offers for the product, sorts them by `priceInPaise` ascending, designates the first element as `lowestOffer`, and calculates savings relative to maximum retail price (MRP).

INTERMEDIATE **What does `lean()` do in Mongoose queries?**

It bypasses the instantiation of full Mongoose document instances with getters, setters, and change tracking, returning plain JavaScript objects. This reduces memory usage and speeds up read queries by 2x to 5x.

INTERMEDIATE **How do you perform text search in MongoDB without external engines like Elasticsearch?**

By creating a text index: `productSchema.index({ name: 'text', description: 'text' })`. Queries then use `{ $text: { $search: query } }` for stemming and keyword matching.

INTERMEDIATE **What is an Upsert operation in MongoDB?**

An update operation that updates matching documents or inserts a new document if no match is found (`{ upsert: true }`), preventing race conditions between check-and-insert.

INTERMEDIATE **How does `seedFixtures.js` ensure idempotency?**

Before inserting seed data, it checks for existing product slugs or uses `findOneAndUpdate` with `{ upsert: true }` rather than blind `insertMany`, allowing it to run repeatedly without duplicate key errors.

INTERMEDIATE **What is the purpose of `$facet` in MongoDB aggregation pipelines?**

It allows executing multiple aggregation pipelines within a single stage on the same input documents, enabling simultaneous retrieval of paginated results and total filter counts.

INTERMEDIATE **How does MongoDB handle write transactions across multiple documents?**

Through multi-document ACID transactions using `session.startTransaction()`, `session.commitTransaction()`, and `session.abortTransaction()` across replica sets.

INTERMEDIATE **What is a TTL (Time-To-Live) index and where is it used in PriceRadar?**

A TTL index automatically removes documents after a specified duration. In PriceRadar, the `OtpChallenge` schema uses a TTL index on `expiresAt` so expired verification codes are purged automatically.

INTERMEDIATE **How does indexing affect write performance in MongoDB?**

While indexes drastically accelerate read queries, every insert, update, or delete must also update all associated index B-trees, slightly increasing write latency and storage overhead.

INTERMEDIATE **What is the difference between `$set` and directly replacing a document in Mongoose?**

`$set` updates only the specified fields while preserving existing document attributes. Direct replacement overwrites the entire document except for `_id`.

INTERMEDIATE **Why is `select('-passwordHash')` important when returning User documents?**

It explicitly projects out the sensitive password hash at the database level so the hash is never loaded into memory or serialized in API responses.

ADVANCED **How would you scale PriceRadar's database to support 10 million products?**

1) Implement database Sharding using a hash-based shard key on `_id` or range-based on `category`. 2) Provision read replicas to offload read-heavy catalog traffic. 3) Cache frequent catalog queries in Redis.

ADVANCED **How does the WiredTiger storage engine handle concurrency and compression?**

WiredTiger uses document-level concurrency control with multi-version concurrency control (MVCC) and snappy/zlib block compression, avoiding table-level locking bottlenecks.

ADVANCED **How would you implement historical price aggregations (e.g., 30-day moving average)?**

Using MongoDB's aggregation pipeline: `$match` observations from the last 30 days, `$group` by day or product, and calculate `$avg: '$priceInPaise'` and `$min: '$priceInPaise'`.

ADVANCED **What are the advantages of MongoDB Atlas compared to self-hosted MongoDB?**

Atlas provides automated multi-region replication, continuous backups, point-in-time recovery, automatic patching, integrated monitoring alerts, and enterprise-grade VPC peering.

ADVANCED **How do you handle schema migrations in Mongoose when adding new required fields?**

Provide schema `default` values for backward compatibility, and run idempotent migration scripts using `updateMany({ field: { $exists: false } }, { $set: { field: defaultValue } })`.

ADVANCED **Explain the CAP theorem trade-offs made by MongoDB.**

MongoDB defaults to Consistency and Partition tolerance (CP). In network partitions, it elects a primary to ensure consistent writes, briefly sacrificing availability during elections.

ADVANCED **How do you detect slow queries in MongoDB Atlas?**

Using the Atlas Performance Advisor and Profiler, which logs operations exceeding the `slowOpThresholdMs` (default 100ms) with execution plan details.

ADVANCED **What is Covered Query in MongoDB?**

A query where all fields requested in the query criteria and returned in the projection are part of an index, allowing MongoDB to return results directly from the index without reading actual documents.

ADVANCED **How do you prevent MongoDB connection pool exhaustion under high traffic?**

By configuring `maxPoolSize` (default 100) in the connection string, reusing Mongoose connections globally, and closing idle connections in test environments.

ADVANCED **How would you implement soft deletes in Mongoose?**

Add `isDeleted: { type: Boolean, default: false }` and `deletedAt: Date` fields, combined with a Mongoose pre-find middleware hook: `this.where({ isDeleted: false })`.

CODE **In `server/src/utils/money.js`, what does `Math.round(rupees * 100)` do?**

It converts fractional rupee values to integer Paise while rounding to the nearest integer to discard floating-point dust.

CODE **In `server/src/models/Offer.js`, what does `offerSchema.index({ product: 1, retailer: 1 }, { unique: true })` achieve?**

It prevents duplicate offers for the same retailer under a given product, enforcing idempotent price updating.

CODE **In `server/src/models/Product.js`, what does `trim: true` on `name` do?**

Mongoose strips any leading or trailing whitespace from the product name before saving to the database.

CODE **What fields are stored in `PriceObservation`?**

`product` (ObjectId ref), `retailer` (String enum), `priceInPaise` (Number), and `recordedAt` (Date).

CODE **In `server/src/controllers/productsController.js`, how are products retrieved with their lowest offer?**

It queries `Product.find()`, queries matching `Offer` records sorted by `priceInPaise: 1`, and attaches the lowest offer to each product payload.

CODE **What is the default sort order in `productsController.getAllProducts`?**

`createdAt: -1`, sorting newly added products to the top of the catalog.

CODE **In `server/src/jobs/seedFixtures.js`, what sample products are seeded?**

Flagship consumer electronics: Apple iPhone 15, MacBook Air M2, Sony WH-1000XM5 headphones, and Samsung Galaxy S24.

CODE **How does `priceComparison.js` handle out-of-stock offers?**

It filters out offers where `inStock === false` before calculating lowest price and target alert triggers.

CODE **In `server/src/models/WatchlistItem.js`, what compound index ensures a user doesn't add the same product twice?**

`watchlistItemSchema.index({ user: 1, product: 1 }, { unique: true })`.

CODE **How does `disconnectDB()` verify clean disconnection in test teardowns?**

It checks `mongoose.connection.readyState !== 0` before calling `await mongoose.connection.close()`.

Part 4: Vamshi — Retailer Integrations, Multi-Source Web Scraping & Cloud Deployment

Specialization: Python Worker, Scrapling Stealth Engine, Apify Cloud Actors, eBay Browse API, Docker, Render PaaS

4.1 Assigned Responsibilities & Deliverables

Vamshi engineered the data collection infrastructure across major Indian e-commerce platforms, third-party API integrations, and production cloud deployment. His deliverables include the Python Scrapling browser worker for anti-bot bypass, the Apify cloud actor integration for Amazon and Flipkart, the eBay official Browse API client, Docker containerization, and production PaaS orchestration on Render.

4.2 Core Code Breakdown & Snippets

A. Python Scrapling Stealth Scraping Worker (`worker/main.py` & `worker/extractor.py`)

FastAPI worker utilizing Scrapling with stealth browser fingerprinting to extract dynamic product prices.

```python
# worker/main.py - High-Performance Extraction Microservice
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, HttpUrl
from extractor import scrape_retailer_product

app = FastAPI(title="PriceRadar Scraper Worker")

class ScrapeRequest(BaseModel):
    url: HttpUrl
    retailer: str

@app.post("/scrape")
async def scrape_endpoint(req: ScrapeRequest):
    try:
        result = scrape_retailer_product(str(req.url), req.retailer)
        if not result or result.get("priceInPaise") is None:
            raise HTTPException(status_code=422, detail="Price extraction failed")
        return {"success": True, "data": result}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

B. Official eBay Browse API Client with OAuth 2.0 (`server/src/services/scrapers/ebay.js`)

Integrates with eBay's official REST API using application client credentials grant tokens.

```javascript
// server/src/services/scrapers/ebay.js - OAuth Token Management & Search
const axios = require('axios');
let cachedToken = null;
let tokenExpiresAt = 0;

async function getEbayToken() {
  if (cachedToken && Date.now() < tokenExpiresAt - 60000) {
    return cachedToken;
  }
  const creds = Buffer.from(`${process.env.EBAY_CLIENT_ID}:${process.env.EBAY_CLIENT_SECRET}`).toString('base64');
  const { data } = await axios.post(
    'https://api.ebay.com/identity/v1/oauth2/token',
    'grant_type=client_credentials&scope=https://api.ebay.com/oauth/api_scope',
    { headers: { 'Authorization': `Basic ${creds}`, 'Content-Type': 'application/x-www-form-urlencoded' } }
  );
  cachedToken = data.access_token;
  tokenExpiresAt = Date.now() + data.expires_in * 1000;
  return cachedToken;
}
```

4.3 Vamshi's Presentation Scripts

2-MINUTE PRESENTATION SCRIPT (SPOKEN IN REVIEW)

"Good morning evaluators. I am Vamshi, and I built the multi-source retailer data collection pipeline and cloud deployment architecture for PriceRadar. Extracting accurate, live pricing from e-commerce giants like Amazon and Flipkart is challenging due to aggressive anti-scraping systems like Cloudflare and Akamai. To overcome this, I engineered a hybrid extraction model: a Python worker utilizing the Scrapling stealth framework with browser fingerprint randomization, backed by managed Apify cloud actors for reliable failover. For global platforms, I integrated directly with eBay's official REST Browse API using OAuth 2.0 client credential flows with in-memory token caching. I containerized the entire ecosystem with Docker and orchestrated continuous production deployments across Render PaaS and MongoDB Atlas."

5-MINUTE VIVA TECHNICAL DEFENSE SCRIPT

"When building an e-commerce price monitoring engine, web scraping reliability is the single most vulnerable point of failure. Traditional headless browsers like standard Puppeteer or Selenium are easily flagged by Cloudflare and Datadome bot detection because they leak navigator properties like `navigator.webdriver = true`. To solve this, I designed a multi-tiered data acquisition pipeline. First, I built an isolated Python microservice using FastAPI and Scrapling. Scrapling patches browser fingerprint leaks, handles JavaScript execution, and mimics human mouse and scrolling behaviors. Second, if local scraping encounters rate limits or CAPTCHAs, our Node.js backend automatically fails over to Apify cloud actors, which rotate residential proxy pools across India to retrieve clean HTML without IP blocks. Third, for structured platforms, I avoided scraping entirely by implementing the official eBay Browse API: we acquire OAuth 2.0 access tokens, cache them in memory with a 60-second safety margin before expiration, and parse live item prices converted to INR. Finally, for cloud deployment, I authored Dockerfiles and `render.yaml` manifests to automate builds, handle environment secrets securely, and ensure health endpoints keep services operational."

4.4 Vamshi's 50 Viva Voce Questions & Answers

BASIC **What is Web Scraping and why is it necessary for PriceRadar?**

Web scraping is the programmatic extraction of data from websites. Because retailers like Amazon and Flipkart do not offer open public price APIs, scraping is necessary to capture live pricing.

BASIC **What is an API and how does it differ from web scraping?**

An Application Programming Interface provides structured, authorized data endpoints (like JSON). Scraping extracts data directly from raw HTML documents rendered for humans.

BASIC **What is Apify?**

Apify is a cloud web scraping and automation platform providing managed scrapers (Actors) equipped with residential proxy rotation and automated browser rendering.

BASIC **What is Scrapling?**

Scrapling is an advanced Python web scraping library that provides high-speed extraction, anti-detect browser capabilities, and stealth fingerprint masking.

BASIC **What is OAuth 2.0 Client Credentials Grant?**

A server-to-server authentication grant where an application exchanges its Client ID and Client Secret for an access token without requiring user interaction.

BASIC **What is Docker and why is it used?**

Docker is an open containerization platform that packages an application and all its dependencies into an isolated container image, guaranteeing consistent execution across development and production.

BASIC **What is a `Dockerfile`?**

A text document containing sequential instructions to assemble a Docker container image (base image, dependencies, environment setup, and start command).

BASIC **What is Render PaaS?**

Render is a cloud Platform-as-a-Service that automatically builds, deploys, and manages web apps, background workers, and databases directly from Git repositories.

BASIC **What is a User-Agent string?**

An HTTP request header identifying the browser type, operating system, and rendering engine to the web server.

BASIC **What is a Proxy Server?**

An intermediary server that routes web requests from client to target web servers, hiding the original client IP address.

BASIC **What is a Residential Proxy vs a Datacenter Proxy?**

Datacenter proxies originate from cloud servers (AWS, DigitalOcean) and are easily flagged by anti-bot systems. Residential proxies route through genuine home ISP connections, making them look like ordinary consumer traffic.

BASIC **What is CAPTCHA and how does it impact scraping?**

Completely Automated Public Turing test to tell Computers and Humans Apart. Anti-bot systems serve CAPTCHAs when suspicious automated traffic is detected, halting unmanaged scraping requests.

BASIC **What is FastAPI?**

A modern, high-performance Python web framework for building APIs with Python 3.8+ based on standard Python type hints, built on Starlette and Pydantic.

BASIC **What is the purpose of `requirements.txt` in Python?**

It lists all Python package dependencies and version pins required to run the application, installed via `pip install -r requirements.txt`.

BASIC **What is an HTTP 429 Status Code?**

HTTP 429 Too Many Requests indicates that the client has sent too many requests in a given amount of time, signaling that rate limits have been triggered.

INTERMEDIATE **How do anti-bot systems detect headless browsers?**

By inspecting navigator properties (`navigator.webdriver === true`), automated screen dimensions, lack of WebGL vendor tags, missing audio codecs, and inhuman interaction speeds.

INTERMEDIATE **How does Scrapling evade anti-bot detection?**

It patches JavaScript prototypes, masks `navigator.webdriver`, spoofs canvas and WebGL fingerprints, injects realistic headers, and simulates human mouse trajectories.

INTERMEDIATE **How does the eBay integration manage OAuth token expiration?**

Tokens are cached in memory alongside an expiration timestamp (`tokenExpiresAt`). Subsequent calls reuse the cached token until 60 seconds before expiration, avoiding redundant authentication requests.

INTERMEDIATE **How does the scraper handle currency conversion from USD to INR for eBay items?**

If eBay returns prices in foreign currencies, our conversion layer queries current exchange rate multipliers or extracts converted INR values directly provided in the localized eBay Browse response.

INTERMEDIATE **What is the purpose of the 4-tier scraper failover architecture?**

1) Fast direct HTTP parsing. 2) Local Scrapling stealth worker. 3) Managed Apify cloud actor with residential proxies. 4) Cached/previous observation fallback. This guarantees high scraping uptime.

INTERMEDIATE **How does `render.yaml` configure multi-service deployments?**

It is an Infrastructure-as-Code manifest defining the Node.js API web service, static frontend site, Python worker, environment variable mappings, and build commands in a single versioned file.

INTERMEDIATE **What is a Docker multi-stage build and why is it beneficial?**

It uses multiple `FROM` instructions in a single Dockerfile to separate the build environment (compilers, build tools) from the final slim runtime image, reducing image size by up to 80%.

INTERMEDIATE **How do you extract pricing from unstructured retailer HTML?**

Using priority selector fallbacks: we test multiple CSS selectors (`.a-price-whole`, `#priceblock_ourprice`), clean out currency symbols and commas via regex (`/[^0-9]/g`), and parse into integers.

INTERMEDIATE **How does the Python worker communicate with the Node.js backend?**

Over an internal HTTP REST interface: the Node backend sends a `POST /scrape` request with the target URL and retailer enum, and the Python worker responds with structured JSON data.

INTERMEDIATE **What is IP rotation and how is it implemented?**

Rotating outgoing IP addresses across a pool of proxy nodes with every request, preventing any single IP from triggering rate-limit blocks.

INTERMEDIATE **How do you handle robots.txt and ethical scraping limits?**

By respecting sensible request frequencies (e.g., 4-hour monitoring cycles rather than aggressive per-second polling) and querying official APIs whenever available.

INTERMEDIATE **What is the cold start problem on Render's free tier and how did you resolve it?**

Free tier services spin down after 15 minutes of inactivity, causing 30-50s latency on wake-up. Our GitHub Actions workflow sends an initial `/api/health` ping before triggering jobs to ensure services are warm.

INTERMEDIATE **How do you debug failed scraping attempts in production?**

By logging HTTP response status codes, saving HTML dumps or screenshots upon extraction failure to cloud storage, and monitoring error metrics in Datadog/Render logs.

INTERMEDIATE **What is CORS and why does the Python worker need CORS configuration?**

If the frontend or external test runners communicate with the worker, the worker must return `Access-Control-Allow-Origin` headers to prevent browser security blocks.

INTERMEDIATE **How does Pydantic validate request payloads in FastAPI?**

Pydantic parses data types from Python type annotations, coercing inputs and returning structured HTTP 422 Unprocessable Entity errors when required fields are missing or invalid.

ADVANCED **How would you scrape Single Page Applications (SPAs) that render entirely on the client?**

By running a headless browser that waits for network idle and specific DOM selectors, or by inspecting network traffic in DevTools to identify internal backend JSON APIs and querying them directly.

ADVANCED **How do you bypass Akamai Bot Manager or Cloudflare Turnstile?**

By using stealth browser automation with TLS fingerprint spoofing (JA3/JA4 fingerprints) so the client SSL handshake matches legitimate consumer browsers like Chrome on Windows.

ADVANCED **What is TLS Fingerprinting (JA3/JA4)?**

A method to identify the client application based on parameters in the SSL/TLS Client Hello packet (cipher suites, extensions, elliptic curves). Standard Python `requests` has a distinct JA3 signature easily blocked by Cloudflare.

ADVANCED **How would you scale scraping to 100,000 products per hour?**

Implement a distributed task queue (Celery with RabbitMQ/Redis), distribute jobs across a cluster of stateless worker containers, and route traffic through a pool of rotating residential proxies.

ADVANCED **What legal considerations govern web scraping (e.g., hiQ Labs v. LinkedIn)?**

Publicly available, unauthenticated data is generally permissible to access under US precedent, provided the scraper does not bypass paywalls, breach terms with authenticated accounts, or degrade server operations.

ADVANCED **How does Docker container networking work on Render?**

Render assigns each container a private hostname on an isolated internal network, allowing the Node backend to query the Python worker without exposing worker ports to the public internet.

ADVANCED **How do you handle dynamic price changes caused by geo-location/PIN codes?**

By sending specific delivery postal codes in cookie headers or query parameters (e.g., setting delivery location cookies) before reading product price elements.

ADVANCED **What is the difference between synchronous and asynchronous HTTP requests in Python?**

Synchronous libraries like `requests` block the thread during network latency. Asynchronous libraries like `httpx` or `aiohttp` yield control to the asyncio event loop, allowing thousands of concurrent requests.

ADVANCED **How do you handle CAPTCHA solving programmatically if stealth fails?**

By integrating automated solving APIs (e.g., 2Captcha, Anti-Captcha) where challenge tokens are submitted to solving networks and response keys injected back into the form.

ADVANCED **How do you implement health check probes for zero-downtime deployments?**

Define an `/api/health` route that checks database connectivity and worker status. Render queries this endpoint before routing traffic to the newly deployed container.

CODE **In `worker/main.py`, what status code is returned if `priceInPaise` is missing?**

HTTP 422 Unprocessable Entity, with `{ detail: 'Price extraction failed' }`.

CODE **In `server/src/services/scrapers/ebay.js`, what does `Date.now() < tokenExpiresAt - 60000` do?**

It checks if the cached OAuth token is still valid with a 60-second safety cushion before attempting a network refresh.

CODE **How does `extractor.py` strip unwanted characters from price strings?**

Using regular expressions: `re.sub(r'[^0-9.]', '', price_text)` to strip commas, currency symbols, and whitespace.

CODE **What endpoint does `server/src/services/scrapers/apify.js` call?**

It initiates Apify actor execution via `POST https://api.apify.com/v2/acts/{actorId}/runs?token={token}`.

CODE **In `Dockerfile`, what base image is utilized for the Node service?**

`node:18-alpine` or `node:20-alpine`, selected for minimal vulnerability footprint and small image size.

CODE **How does `render.yaml` define environment variables for production?**

Under the `envVars` block, specifying keys and mapping values either from secrets or internal service hostnames.

CODE **In `worker/requirements.txt`, what are the core packages installed?**

`fastapi`, `uvicorn`, `scrapling`, `pydantic`, and `httpx`.

CODE **How does the eBay Browse API request specify search keywords?**

Via the `q` query parameter: `/buy/browse/v1/item_summary/search?q={encodeURIComponent(query)}&limit=5`.

CODE **What does the `inStock` boolean in the scraped data payload indicate?**

Whether the retailer has active inventory available for purchase or displays 'Currently Unavailable / Out of Stock'.

CODE **How does the scraper identify if a product is an Amazon Lightning Deal?**

By checking for the existence of deal badge classes like `.dealBadge` or `#dealprice_savings` in the product DOM.

Part 5: Ritesh — Scheduled Monitoring Engine, Alert Logic, Notifications & CI/CD

Specialization: Distributed Locking, Background Jobs, Brevo Email API, Telegram Bot API, GitHub Actions CI/CD

5.1 Assigned Responsibilities & Deliverables

Ritesh engineered the core asynchronous monitoring cycle, distributed synchronization guards, price drop alert evaluation algorithms, multi-channel notification dispatchers, and GitHub Actions automation workflows. His deliverables include `priceMonitor.js`, atomic distributed leasing (`MonitorLock`), notification adapters for Brevo and Telegram, alert evaluation logic with notification deduplication, and the complete 4-hour automation pipeline.

5.2 Core Code Breakdown & Snippets

A. Atomic Distributed Lock & Execution Lease (`server/src/models/MonitorLock.js` & `priceMonitor.js`)

Guarantees that exactly one monitoring cycle runs across multiple server instances using atomic MongoDB operations.

```
// server/src/models/MonitorLock.js - Atomic Lease Acquisition
const mongoose = require('mongoose');

const monitorLockSchema = new mongoose.Schema({
  lockKey: { type: String, required: true, unique: true, default: 'price_monitor_lock' },
  isLocked: { type: Boolean, default: false },
  lockedBy: { type: String, default: null },
  leaseExpiresAt: { type: Date, default: null }
}, { timestamps: true });

monitorLockSchema.statics.acquireLease = async function (workerId, leaseDurationMs = 300000) {
  const now = new Date();
  const leaseExpiresAt = new Date(now.getTime() + leaseDurationMs);

  // Atomic findOneAndUpdate passes only if unlocked OR existing lease has expired
  const lock = await this.findOneAndUpdate(
    {
      lockKey: 'price_monitor_lock',
      $or: [
        { isLocked: false },
        { leaseExpiresAt: { $lt: now } }
      ]
    },
    {
      $set: { isLocked: true, lockedBy: workerId, leaseExpiresAt }
    },
    { new: true, upsert: true }
  );
  return !!lock;
};
```

B. Notification Deduplication Key (`server/src/models/NotificationEvent.js`)

Prevents spamming users with duplicate alerts by enforcing unique compound constraints on alert notifications.

```
// server/src/models/NotificationEvent.js - Deduplication Schema
const notificationEventSchema = new mongoose.Schema({
  alert: { type: mongoose.Schema.Types.ObjectId, ref: 'Alert', required: true },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  channel: { type: String, enum: ['EMAIL', 'TELEGRAM'], required: true },
  priceInPaise: { type: Number, required: true },
  dedupKey: { type: String, required: true, unique: true },
  status: { type: String, enum: ['PENDING', 'SENT', 'FAILED'], default: 'PENDING' }
}, { timestamps: true });
```

```
// Format: `alertId_priceInPaise_YYYY-MM-DD` prevents multiple alerts on same day for same price
notificationEventSchema.statics.generateDedupKey = function(alertId, priceInPaise) {
  const date = new Date().toISOString().slice(0, 10);
  return `${alertId}_${priceInPaise}_${date}`;
};
```

5.3 Ritesh's Presentation Scripts

2-MINUTE PRESENTATION SCRIPT (SPOKEN IN REVIEW)

"Good morning evaluators. I am Ritesh, and I engineered the autonomous monitoring engine, distributed concurrency locks, price drop alert engine, and CI/CD pipelines for PriceRadar. Our platform tracks watched products on a strict 4-hour cycle. To guarantee that multiple server instances or simultaneous trigger requests never execute overlapping scrape cycles, I implemented a distributed atomic database lease using MongoDB's `findOneAndUpdate` with lease expiration recovery. When live prices drop below user-configured targets, our alert evaluator triggers multi-channel notifications via Brevo transactional emails and Telegram Bot webhooks. To prevent alert spam, I engineered a deterministic deduplication key system. I also created our automated GitHub Actions workflow which executes every 4 hours, awakens our cloud instance, initiates the monitoring run, and monitors progress to completion."

5-MINUTE VIVA TECHNICAL DEFENSE SCRIPT

"In designing PriceRadar's automated monitoring engine, the paramount challenge was preventing race conditions, orphaned processes, and notification storms. In a distributed cloud environment like Render or AWS, multiple container instances might receive trigger requests concurrently. A naive check like `if (isRunning) return` suffers from a classic time-of-check to time-of-use (TOCTOU) race condition. To solve this, I designed `MonitorLock` using an atomic MongoDB `findOneAndUpdate` operation. A worker can only acquire the lease if `isLocked` is false or the previous lease timestamp has passed, guaranteeing mutual exclusion. When the lease is secured, the endpoint immediately returns HTTP 202 Accepted with a unique `runId`, decoupling the client from the long-running background job. During execution, we iterate through active watchlist products, gather fresh prices across retailers, and evaluate active alert thresholds. If a price satisfies the alert criteria, we dispatch notifications via Brevo Email API and Telegram Bot API. To prevent duplicate notifications if a job retries, we generate a unique `dedupKey` combining the alert ID, price, and current date. Finally, our entire test architecture is secured through GitHub Actions CI: 25 test suites containing 263 tests run on every pull request, verifying zero open handles and deterministic execution."

5.4 Ritesh's 50 Viva Voce Questions & Answers

BASIC **What is a Cron Job and where is it configured in PriceRadar?**

A cron job is a time-based task scheduler. In PriceRadar, it is configured in `.github/workflows/monitor.yml` using the schedule `0 */4 * * *` (every 4 hours).

BASIC **What is CI/CD?**

Continuous Integration (automating building and testing code on every push) and Continuous Deployment (automating release to production servers).

BASIC **What is GitHub Actions?**

GitHub's native automation platform that allows running workflows on virtual runners triggered by repository events (push, pull_request, schedule).

BASIC **What is Brevo (formerly Sendinblue)?**

A cloud email service provider providing REST APIs for sending reliable transactional emails.

BASIC **What is Telegram Bot API?**

An HTTP-based interface for developers to build bots that send automated text messages, alerts, and rich media to Telegram users and channels.

BASIC **What is an Asynchronous Background Job?**

A long-running process executed outside the main HTTP request/response cycle, allowing APIs to respond immediately with HTTP 202 Accepted without keeping connections open.

BASIC **What is a Distributed Lock?**

A synchronization mechanism that ensures only one process or server instance across a distributed network can execute a critical section at any given time.

BASIC **What is an HTTP 202 Accepted status code?**

It indicates that the request has been accepted for processing, but processing has not been completed, ideal for triggering background jobs.

BASIC **What is a Webhook?**

An automated HTTP callback triggered by an event, allowing one system to deliver real-time notifications to another.

BASIC **What is Jest?**

A comprehensive JavaScript testing framework that provides test runners, assertion libraries, and mock support.

BASIC **What is a Unit Test vs an Integration Test?**

A unit test isolates and tests an individual function or class. An integration test verifies that multiple units (e.g., Express routes, middleware, and database) function together correctly.

BASIC **What is an Open Handle in Jest?**

An active timer, socket, or open database connection that prevents the Node.js process from cleanly terminating after tests finish.

BASIC **What is a Smoke Test?**

A preliminary test suite that verifies critical system functionality (e.g., health check, DB connection) before running deeper tests.

BASIC **What is Idempotence in software engineering?**

The property of an operation where executing it multiple times yields the exact same outcome as executing it once.

BASIC **What is an Environment Variable?**

A dynamic key-value pair configured outside the source code to provide configuration and secrets to running applications.

INTERMEDIATE **Explain the Time-of-Check to Time-of-Use (TOCTOU) race condition in price monitoring.**

If instance A checks `isLocked` (false) and before it sets it to true, instance B also checks `isLocked` (false), both instances believe they hold the lock and execute simultaneously. An atomic operation eliminates this gap.

INTERMEDIATE **How does `MonitorLock.acquireLease` solve the TOCTOU race condition?**

It uses MongoDB's atomic `findOneAndUpdate` with a query filtering for `{ $or: [{ isLocked: false }, { leaseExpiresAt: { $lt: now } }] }`. MongoDB's internal locking ensures only one update succeeds.

INTERMEDIATE **What happens if a backend instance crashes while holding the lock?**

The lease includes an expiration timestamp (`leaseExpiresAt = now + 5 mins`). After 5 minutes, subsequent runs treat the lock as expired and automatically acquire the lease, recovering gracefully from crashes.

INTERMEDIATE **Why does `POST /api/monitor/run` return HTTP 202 Accepted instead of waiting for completion?**

Scraping multiple retailers takes 30 to 120 seconds. HTTP clients (and Render's 30s timeout) would drop the connection. Returning HTTP 202 with a `runId` allows the client to poll `/api/monitor/status/:runId` asynchronously.

INTERMEDIATE **How does the deduplication key in `NotificationEvent` prevent email spam?**

The `dedupKey` is composed of `${alertId}_${priceInPaise}_${date}` backed by a unique index. Attempting to insert a duplicate event on the same day for the same price throws an E11000 duplicate key error, preventing repeated sends.

INTERMEDIATE **How does `.github/workflows/monitor.yml` handle Render free-tier cold starts?**

Step 1 sends a `curl` request to `https://priceradar-api-02ib.onrender.com/api/health` with retry loops, ensuring the container is fully awake before triggering `POST /api/monitor/run`.

INTERMEDIATE **How do you test notification dispatchers without sending real emails in unit tests?**

Using Jest mocks: `jest.mock('../services/notifications/email')`, allowing assertions like `expect(sendEmail).toHaveBeenCalledWith(...)` without network I/O.

INTERMEDIATE **What is the role of `MonitorRun` model in tracking job history?**

It stores execution telemetry: `runId`, `startedAt`, `finishedAt`, `status` ('running', 'completed', 'failed'), `productsScanned`, `alertsTriggered`, and error logs.

INTERMEDIATE **How does Telegram Bot authenticate notification requests?**

Via an authorization token embedded in the URL: `https://api.telegram.org/bot/sendMessage`, providing authenticated message delivery to the configured `TELEGRAM_CHAT_ID`.

INTERMEDIATE **Why was `disconnectDB()` introduced into Jest test teardowns?**

Mongoose keeps connection pools open. Calling `await disconnectDB()` in `afterAll` closes the socket connections, allowing Jest to exit without `--forceExit`.

INTERMEDIATE **What is the difference between `MONITORING_ENABLED=true` and `false`?**

When `false`, scheduled monitoring skips external web scraping, allowing test suites to verify lock management and error handling without external network dependencies.

INTERMEDIATE **How does `alertEvaluator.js` determine if an alert should fire?**

It compares the lowest current `priceInPaise` against the user's `targetPriceInPaise`. If `currentPrice <= targetPrice`, the condition is satisfied and passed to the dispatcher.

INTERMEDIATE **What happens if Brevo email API fails during alert dispatch?**

The error is caught in a `try/catch` block, logged to `NotificationEvent.status = 'FAILED'`, and execution continues so Telegram alerts and remaining user notifications are not aborted.

INTERMEDIATE **How does `ci.yml` run automated tests on GitHub Actions?**

It provisions an `ubuntu-latest` runner, spins up a MongoDB service container, installs dependencies, seeds test fixtures, and executes `npm test -- --runInBand`.

INTERMEDIATE **Why is `--runInBand` used in CI test execution?**

It forces Jest to execute test suites sequentially in a single process rather than spawning worker pools, preventing race conditions on shared MongoDB test databases.

ADVANCED **How would you adapt the distributed lock for a cluster with 50 backend nodes?**

While MongoDB's atomic findOneAndUpdate scales well up to hundreds of nodes, Redis with the Redlock algorithm or AWS DynamoDB with conditional writes offers sub-millisecond lease acquisition with lower database load.

ADVANCED **How would you implement exponential backoff retry for failed notifications?**

Store `retryCount` on `NotificationEvent`. On failure, compute delay: $t = 2^{\text{retryCount}} \times 1000\text{ms}$. Schedule retry using BullMQ or a delayed job queue up to a maximum of 5 attempts.

ADVANCED **What is a Dead Letter Queue (DLQ) and how would it apply to PriceRadar?**

A DLQ captures messages or alerts that repeatedly fail after maximum retries, isolating problematic payloads for manual inspection without blocking the active queue.

ADVANCED **How do you ensure zero-downtime database migrations in a CI/CD pipeline?**

Follow the expand-and-contract pattern: 1) Deploy code that supports both old and new schema. 2) Migrate data in background. 3) Deploy code using only the new schema. 4) Drop old fields.

ADVANCED **How would you monitor background job health in real time?**

Expose Prometheus metrics (`monitor_run_duration_seconds`, `monitor_alerts_sent_total`) and visualize them in Grafana dashboards, with PagerDuty alerts triggered if a cycle fails.

ADVANCED **How does Brevo handle email deliverability and avoid Spam folders?**

By validating SPF (Sender Policy Framework), DKIM (DomainKeys Identified Mail), and DMARC DNS records on the sending domain to authenticate server identity.

ADVANCED **What are the security implications of storing Bot Tokens in CI/CD secrets?**

If exposed, attackers could take control of the bot to spam users or read channel messages. Secrets must be restricted to protected branches and never echoed in workflow logs.

ADVANCED **How would you scale price monitoring from 100 to 1,000,000 products?**

Partition products into batches using BullMQ or Kafka topics, distribute batches across auto-scaling worker nodes, and utilize priority queues for products with active user alerts.

ADVANCED **How does MongoDB's write concern impact distributed lock safety?**

We use `{ writeConcern: { w: 'majority' } }` to ensure lock acquisition is committed across a majority of replica set nodes, preventing stale lock re-acquisitions after a primary election.

ADVANCED **What is the difference between polling and event-driven architectures for price alerts?**

Polling periodically checks prices at intervals (every 4 hours), while event-driven architectures react to real-time retailer webhook streams (which e-commerce platforms rarely provide publicly).

CODE **In `server/src/models/MonitorLock.js`, what does `upsert: true` accomplish?**

If the lock document `price_monitor_lock` does not yet exist in the collection, MongoDB creates it atomically during the initial lease acquisition.

CODE **In `server/src/services/priceMonitor.js`, how is `workerId` generated?**

`const workerId = 'worker_' + process.pid + '_' + Date.now();`, providing an unambiguous identifier for each running Node instance.

CODE **In `server/src/controllers/monitorController.js`, what does `triggerMonitoring` return?**

HTTP 202 Accepted with `{ success: true, message: 'Monitoring started in background', runId }`.

CODE **In `server/src/models/NotificationEvent.js`, what format is the `dedupKey`?**

`${alertId}_${priceInPaise}_${date}`, bounding deduplication by alert, price, and calendar day.

CODE **In `server/src/services/notifications/email.js`, what library is used to talk to Brevo?**

`@getbrevo/brevo` or Axios calling `https://api.brevo.com/v3/smtp/email` with the `api-key` header.

CODE **In `server/src/services/notifications/telegram.js`, what payload is sent to the Telegram API?**

`{ chat_id, text, parse_mode: 'HTML' }` sent via `POST https://api.telegram.org/bot/sendMessage`.

CODE **In `.github/workflows/monitor.yml`, what is the cron schedule?**

`0 */4 * * *`, which triggers the workflow at minute 0 of every 4th hour UTC.

CODE **What does `server/tests/monitorExecution.test.js` verify?**

It simulates concurrent trigger requests and verifies that only one instance acquires the lock while the other receives HTTP 409 or a duplicate rejection.

CODE **In `priceMonitor.js`, what happens in the `finally` block of the monitoring run?**

It releases the in-process mutex and calls `MonitorLock.releaseLock(workerId)`, ensuring locks are never orphaned on exceptions.

CODE **What does the status polling endpoint `GET /api/monitor/status/:runId` return?**

The latest `MonitorRun` document, including `status`, `productsScanned`, `alertsTriggered`, and duration.

Part 6: Shared Team-Level Viva Defense Bank & Evaluation Strategy

System-Wide Architectural Questions, Trade-Off Discussions, Cloud Constraints, and Panel Defense Strategy

6.1 System-Wide Technical Viva Questions

SHARED VIVA **Explain the high-level architecture of PriceRadar in 60 seconds.**

PriceRadar is a full-stack, distributed price-tracking platform. The frontend is an SPA built in React 18 and Vite. It communicates via Axios over REST with a Node.js/Express backend secured by JWT and Helmet. Data is stored in MongoDB Atlas, with prices stored in integer Paise. Data collection uses a hybrid approach: an official eBay Browse API client, a Python Scrapling stealth worker, and Apify cloud actors. A 4-hour background cycle managed by GitHub Actions cron and atomic MongoDB distributed leases checks prices, evaluates alerts, and sends real-time notifications via Brevo Email and Telegram Bot APIs.

SHARED VIVA **Why did the team choose MongoDB Atlas over PostgreSQL?**

E-commerce product attributes are highly polymorphic across retailers: an electronic item on Amazon has different specifications than on Flipkart. MongoDB's document model handles this variation naturally without complex relational joins or wide, mostly-empty tables. Additionally, MongoDB Atlas offers effortless cloud replication, automated scaling, and rich compound indexing for our sub-millisecond queries.

SHARED VIVA **Why did you store currency as integer Paise instead of standard floating-point Rupees?**

In JavaScript and standard computing, floating-point numbers follow IEEE 754, causing rounding inaccuracies like `0.1 + 0.2 = 0.30000000000000004`. In financial and alert calculations, these microscopic errors could falsely trigger price-drop alerts or corrupt savings calculations. Storing prices as integer Paise (₹1499.50 = 149950 Paise) guarantees absolute mathematical precision.

SHARED VIVA **How does PriceRadar prevent two simultaneous monitoring runs from executing at the same time?**

Through a dual-layer concurrency guard. First, an in-process local mutex blocks concurrent triggers on the same server instance. Second, for multi-instance deployments, we engineered an atomic distributed lease in MongoDB (`MonitorLock`) using `findOneAndUpdate`. It atomically grants the lock only if it is unlocked or the previous lease has expired, preventing time-of-check to time-of-use (TOCTOU) race conditions.

SHARED VIVA **What happens if a backend worker crashes in the middle of a price-monitoring cycle?**

The distributed lock has a finite lease duration (5 minutes). If a process crashes, the lock is never permanently orphaned: when the lease expires, the next scheduled cycle or manual trigger automatically reclaims the lock and records the previous run as interrupted, allowing the system to self-heal without manual database intervention.

SHARED VIVA **How does PriceRadar bypass anti-scraping defenses like Cloudflare and Akamai?**

We use a multi-tiered strategy. For global products, we avoid scraping entirely by querying official eBay Browse REST APIs with OAuth 2.0. For domestic retailers, we use an isolated Python worker with Scrapling, which masks `navigator.webdriver`, randomizes browser fingerprints, and mimics human interaction. For heavy anti-bot challenges, we fail over to managed Apify cloud actors that rotate residential IP pools across India.

SHARED VIVA **How do you handle the 15-minute cold start sleep on Render's free tier?**

Render puts inactive free web services to sleep after 15 minutes. To ensure our scheduled 4-hour monitoring cycle never times out, our GitHub Actions workflow executes an initial wake-up step: it sends health-check pings to `/api/health` with retry loops, ensuring the container is fully awake before issuing the `POST /api/monitor/run` execution request.

SHARED VIVA **Why does `POST /api/monitor/run` return HTTP 202 Accepted instead of waiting for results?**

Scraping multiple products across multiple e-commerce websites takes anywhere from 30 to 120 seconds. Standard HTTP connections and reverse proxies (like Render's 30-second gateway timeout) would drop the connection. Returning HTTP 202 immediately with a unique `runId` decouples execution from client connection lifetimes and allows clients to poll status asynchronously.

SHARED VIVA **How do you prevent users from being spammed with duplicate notifications?**

We generate a deterministic deduplication key in the `NotificationEvent` collection: `${alertId}_${priceInPaise}_${YYYY-MM-DD}`, backed by a unique database index. If a monitoring cycle re-evaluates an existing price drop on the same day, MongoDB rejects the duplicate insertion with code 11000, preventing redundant emails or Telegram messages.

SHARED VIVA **How did you ensure test suite reliability without leaving open handles in Jest?**

Our test suite of 25 suites and 263 tests uses Supertest to test the Express `app` without binding to live TCP ports. In test teardown hooks (`afterAll`), we invoke `disconnectDB()`, which closes Mongoose connection pools cleanly. This allows Jest to exit with status 0 without needing the `--forceExit` flag.

SHARED VIVA **How is user authentication secured from end to end?**

Passwords are encrypted using bcrypt with 10 salt rounds before database storage. Authentication issues an HMAC-SHA256 signed JWT with a 7-day expiration. The client stores the JWT in `localStorage` and attaches it via an Axios interceptor to the `Authorization: Bearer ` header. Express middleware verifies the token signature and loads user context while stripping `passwordHash`.

SHARED VIVA **How would you scale PriceRadar from 100 products to 1,000,000 products?**

1) Replace the direct background loop with a distributed message queue (RabbitMQ or BullMQ with Redis). 2) Shard the MongoDB cluster by product ID. 3) Deploy auto-scaling scraping worker pools in Kubernetes. 4) Cache product catalog views in Redis or Cloudflare CDN. 5) Partition monitoring jobs into priority queues based on user watchlist activity.

SHARED VIVA **What is the difference between client-side debouncing and server-side rate limiting?**

Debouncing is implemented on the frontend to delay sending API calls until the user stops typing (e.g., waiting 300ms in product search). Rate limiting is implemented on the backend (`express-rate-limit`) to reject abusive traffic exceeding threshold limits (e.g., maximum 100 requests per 15 minutes per IP). Both are essential for performance and defense.

SHARED VIVA **What happens if an external retailer goes down or changes its HTML structure?**

Our scraping parser uses defensive CSS selector arrays. If all selectors fail, it logs an extraction error for that specific retailer and sets `inStock: false`, while allowing other retailer offers for that product to update normally. The failure is logged in the `MonitorRun` telemetry record without crashing the monitoring cycle.

SHARED VIVA **How did your team collaborate and manage code versions during development?**

We used Git with feature-branch workflows and GitHub pull requests. Each team member worked in their respective domain (frontend, backend, database, scraping, monitoring). Continuous Integration workflows on GitHub Actions ran our 25 test suites and Vite production build on every PR, ensuring that no broken code was ever merged into the main branch.

SHARED VIVA **What are the key security headers enforced by Helmet.js?**

Helmet sets `Content-Security-Policy` (prevents XSS), `X-Frame-Options: DENY` (prevents clickjacking), `X-Content-Type-Options: nosniff` (prevents MIME sniffing), and `Strict-Transport-Security` (enforces HTTPS), securing the API against common web vulnerabilities.

SHARED VIVA **Why did you build a separate Python microservice instead of running all scraping in Node.js?**

While Node.js excels at asynchronous I/O and REST APIs, Python boasts superior stealth scraping libraries like Scrapling that handle low-level browser fingerprint masking, DOM parsing, and bot evasion more effectively than JavaScript alternatives. Packaging it as a FastAPI microservice maintained clean architectural separation.

SHARED VIVA **What is the environmental footprint and operational cost of PriceRadar?**

PriceRadar was designed to run within zero-cost free-tier cloud limits: Render for backend web services and static frontend hosting, MongoDB Atlas Free Tier for the database, Apify free tier for proxy scraping, Brevo free tier for transactional emails, and GitHub Actions free minutes for scheduled cron automation.

SHARED VIVA **How does PriceRadar protect against SQL/NoSQL injection attacks?**

We use Mongoose schemas which strictly enforce types and strip out unknown fields. We also sanitize user inputs and utilize parameterized Mongoose queries, preventing malicious query operators like `{$gt: ''}` from bypassing authentication.

SHARED VIVA **What would be your first priority if you had an additional month to work on this project?**

1) Native mobile application using React Native. 2) Machine learning price-prediction models (ARIMA / LSTM) that advise users whether to 'Buy Now' or 'Wait for Price Drop' based on seasonal sales patterns. 3) Push notifications via Web Push API and WhatsApp Business API.

SHARED VIVA **How do you test frontend components in isolation?**

Using Vitest and React Testing Library to render components into a virtual DOM, simulate user interactions (clicking buttons, typing text), and assert expected UI state changes.

SHARED VIVA **How does PriceRadar maintain data freshness without overwhelming retailer websites?**

By tracking prices on a 4-hour cycle rather than continuous polling. 4 hours represents an optimal balance: e-commerce price updates typically occur during midnight sales refreshes or flash deals, making 6 checks per day sufficient while preventing retailer IP bans.

SHARED VIVA **What is the role of `server/src/models/OtpChallenge.js`?**

It stores temporary 6-digit verification codes generated during email verification and password resets, equipped with a 10-minute MongoDB TTL index that automatically purges expired codes from the database.

SHARED VIVA **How do you verify that your production deployment matches the tested codebase?**

Our GitHub Actions CI workflow compiles the Vite production build and executes all 263 Jest unit and integration tests. Deployments on Render are configured to auto-deploy exclusively from the verified `main` branch after CI checks pass.

SHARED VIVA **If an interviewer asks: 'What was the hardest bug your team solved?', what is the recommended answer?**

The asynchronous distributed monitoring race condition. Under heavy traffic or overlapping cron triggers, multiple instances could pass the initial `isLocked` check simultaneously (a TOCTOU race), resulting in duplicate scraping cycles, rate-limit bans from retailers, and duplicate email alerts. We solved this by implementing an atomic MongoDB lease (`findOneAndUpdate`) with automatic stale-lease recovery and deterministic deduplication keys.

6.2 Project Viva Voce Defense Strategy & Presentation Tips

Golden Rules for a High-Scoring Project Viva

1. **Own Your Specific Track While Knowing the Big Picture:** When an evaluator asks an architecture question, start with a concise system-wide overview, then smoothly transition into your specific module to demonstrate deep mastery.
2. **Back Statements with Concrete Repository Facts:** Never say "we wrote some tests." Say: "*We wrote 25 Jest test suites containing 263 passing unit and integration tests, verifying zero open handles during teardown.*"
3. **Explain the 'Why' Behind Technical Choices:** Evaluators care about engineering trade-offs. Be ready to explain why you chose integer Paise over floats, MongoDB over MySQL, Vite over CRA, and distributed atomic locks over simple in-memory booleans.
4. **Acknowledge Constraints Honestly:** When asked about free-tier cloud latency, explain how you engineered GitHub Actions wake-up pings to overcome Render's 15-minute container sleep. This shows practical problem-solving.
5. **Demonstrate Code-Level Familiarity:** Be prepared to open any core file and walk through lines of code, explaining function parameters, regex patterns, and error handlers.

Team Handoff Protocol During the Evaluation

If an evaluator asks a question outside your assigned track, practice clean professional handoffs:

- "*While that affects the frontend presentation layer, my teammate Mrunal engineered our UI state and can explain the exact React implementation.*"
- "*That ties directly into our database schema and indexing strategy, which Yaswanth architected.*"
- "*Vamshi handled our Scrapling worker and anti-bot bypass, and can walk through the stealth fingerprinting pipeline.*"
- "*Ritesh built the distributed locking engine and notification dispatchers, and can explain how we prevent race conditions.*"
- "*Rahul authored our JWT security middleware and can explain our authentication and validation chain.*"